



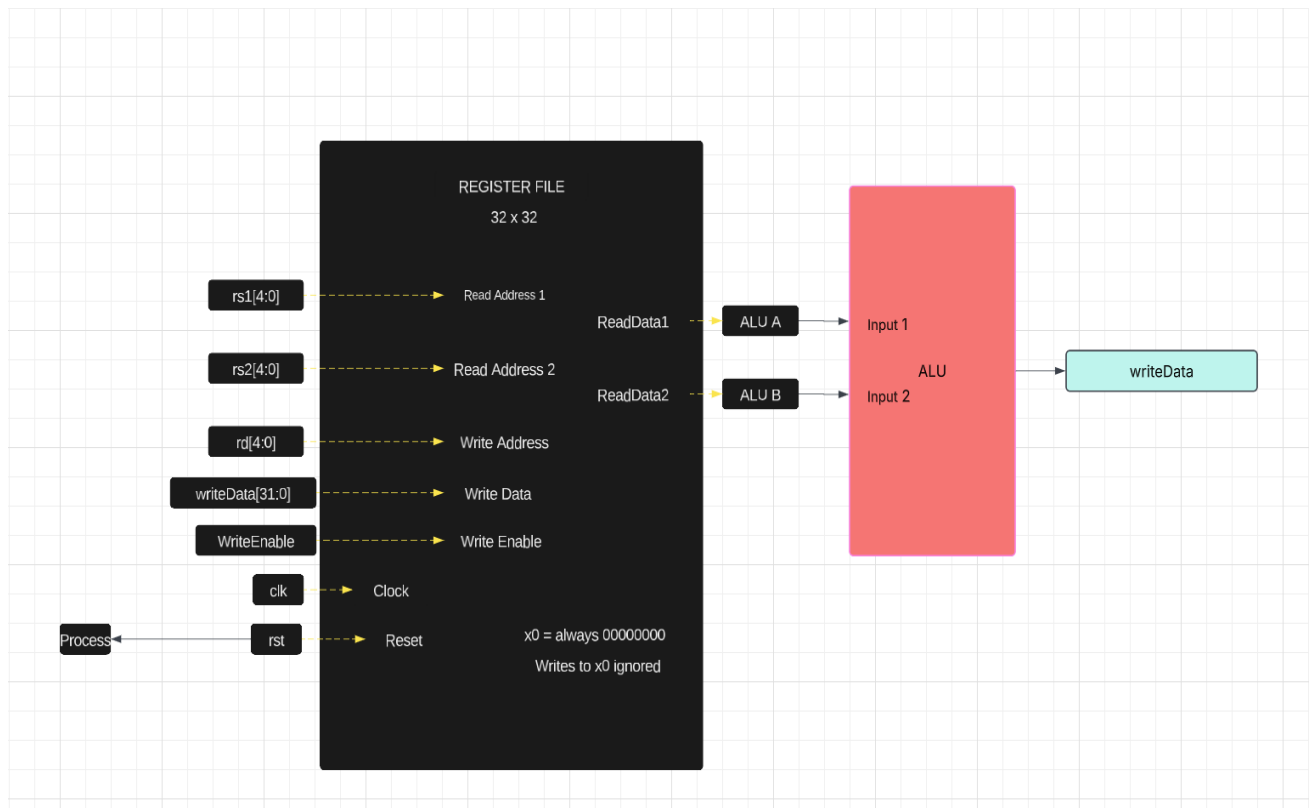
Lab 07 – Worksheet

Name:	ID:	Section: T2
Abdullah Rasool	ar11014	
Arsalan Rashid	ar12008	
Wasay Ali	wa11538	

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Register File Diagram:



Comments

**WRITE:**

WriteData -> selected rd register

only when:

WriteEnable = 1

rd != 0

rising edge of clk

READ:

rs1 -> ReadData1 asynchronous

rs2 -> ReadData2 asynchronous

x0:

reading x0 -> 0

writing x0 -> ignored

Task 2(A & B):

Modules + Testbench:

Part A Design files:**RegisterFile.v**

```
module RegisterFile (  
    input wire clk,  
    input wire rst,  
    input wire WriteEnable,  
  
    input wire [4:0] rs1,  
    input wire [4:0] rs2,  
    input wire [4:0] rd,  
  
    input wire [31:0] WriteData,  
  
    output wire [31:0] ReadData1,  
    output wire [31:0] ReadData2  
);  
  
    reg [31:0] regs [0:31];
```



```
integer i;

// Synchronous write, asynchronous reset
always @(posedge clk or posedge rst) begin
    if (rst) begin
        for (i = 0; i < 32; i = i + 1)
            regs[i] <= 32'b0;
        end
    else begin
        // x0 cannot be written
        if (WriteEnable && (rd != 5'd0))
            regs[rd] <= WriteData;
        end
    end
end

// Asynchronous reads
// x0 always returns zero
assign ReadData1 = (rs1 == 5'd0) ? 32'b0 : regs[rs1];
assign ReadData2 = (rs2 == 5'd0) ? 32'b0 : regs[rs2];

Endmodule
```

ALU.v

```
module ALU (
    input wire [31:0] A,
    input wire [31:0] B,
    input wire [2:0] ALUControl,

    output reg [31:0] Result,
    output wire Zero
);

always @(*) begin
    case (ALUControl)

        3'b000:
            Result = A + B;    // ADD

        3'b001:
            Result = A - B;    // SUB

        3'b010:
```



```
Result = A & B;    // AND

3'b011:
Result = A | B;    // OR

3'b100:
Result = A ^ B;    // XOR

3'b101:
Result = A << B[4:0]; // SLL

3'b110:
Result = A >> B[4:0]; // SRL

3'b111:
Result = A - B;    // BEQ comparison

default:
Result = 32'b0;

endcase
end

assign Zero = (Result == 32'b0);

Endmodule
```

Part B Testbench:

RegisterFile tb.v

```
`timescale 1ns / 1ps

module RegisterFile_tb;

    reg clk;
    reg rst;
    reg WriteEnable;

    reg [4:0] rs1;
    reg [4:0] rs2;
```



```
reg [4:0] rd;

reg [31:0] WriteData;

wire [31:0] ReadData1;
wire [31:0] ReadData2;

RegisterFile uut (
    .clk(clk),
    .rst(rst),
    .WriteEnable(WriteEnable),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd),
    .WriteData(WriteData),
    .ReadData1(ReadData1),
    .ReadData2(ReadData2)
);

// 100 MHz equivalent simulation clock
always #5 clk = ~clk;

task write_register;

    input [4:0] address;
    input [31:0] data;

    begin

        @(negedge clk);

        rd = address;
        WriteData = data;
        WriteEnable = 1'b1;

        @(posedge clk);
        #1;

        WriteEnable = 1'b0;

    end
```



```
endtask
```

```
initial begin
```

```
    clk = 0;  
    rst = 0;
```

```
    WriteEnable = 0;
```

```
    rs1 = 0;  
    rs2 = 0;  
    rd = 0;
```

```
    WriteData = 0;
```

```
    // -----  
    // RESET  
    // -----
```

```
    #2;  
    rst = 1;
```

```
    #10;  
    rst = 0;
```

```
    // -----  
    // TEST 1  
    // Write x5 = DEADBEEF  
    // -----
```

```
    write_register(5'd5, 32'hDEADBEEF);
```

```
    rs1 = 5'd5;
```

```
    #2;
```

```
    if (ReadData1 == 32'hDEADBEEF)  
        $display("PASS: x5 written correctly");  
    else  
        $display("FAIL: x5 incorrect");
```



```
// -----  
// TEST 2  
// Try writing x0  
// -----  
  
write_register(5'd0, 32'hFFFFFFFF);  
  
rs1 = 5'd0;  
  
#2;  
  
if (ReadData1 == 32'h00000000)  
    $display("PASS: x0 remained zero");  
else  
    $display("FAIL: x0 changed");  
  
// -----  
// TEST 3  
// Two simultaneous reads  
// -----  
  
write_register(5'd6, 32'h12345678);  
write_register(5'd7, 32'hCAFEBABE);  
  
rs1 = 5'd6;  
rs2 = 5'd7;  
  
#2;  
  
if ((ReadData1 == 32'h12345678) &&  
    (ReadData2 == 32'hCAFEBABE))  
  
    $display("PASS: simultaneous reads correct");  
  
else  
    $display("FAIL: simultaneous reads incorrect");  
  
// -----  
// TEST 4  
// Overwrite x5  
// -----  
  
write_register(5'd5, 32'hAAAAAAAA);
```



```
rs1 = 5'd5;

#2;

if (ReadData1 == 32'hAAAAAAAA)
    $display("PASS: overwrite successful");
else
    $display("FAIL: overwrite unsuccessful");

// -----
// TEST 5
// Reset
// -----

rst = 1;

#2;

rst = 0;

rs1 = 5'd5;
rs2 = 5'd6;

#2;

if ((ReadData1 == 0) &&
    (ReadData2 == 0))

    $display("PASS: reset cleared registers");

else
    $display("FAIL: reset unsuccessful");

$display("-----");
$display("REGISTER FILE TEST COMPLETE");
$display("-----");

#20;

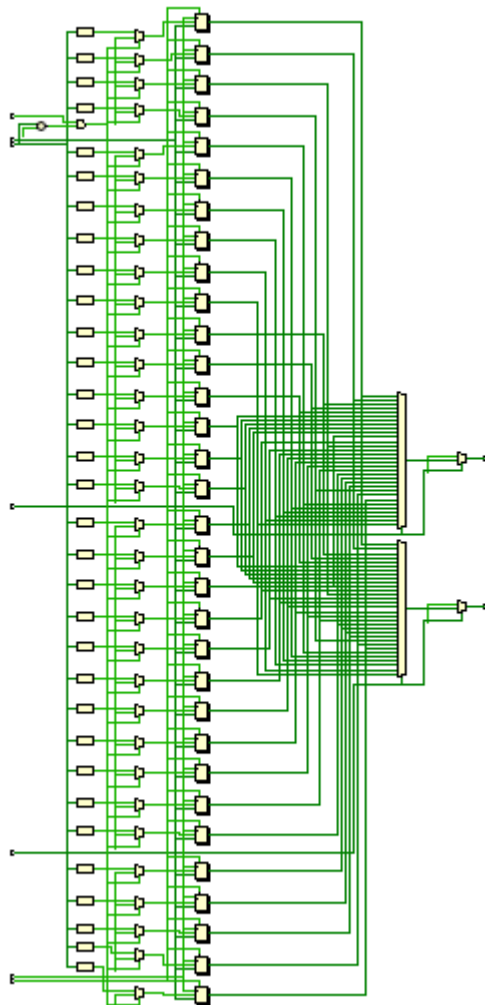
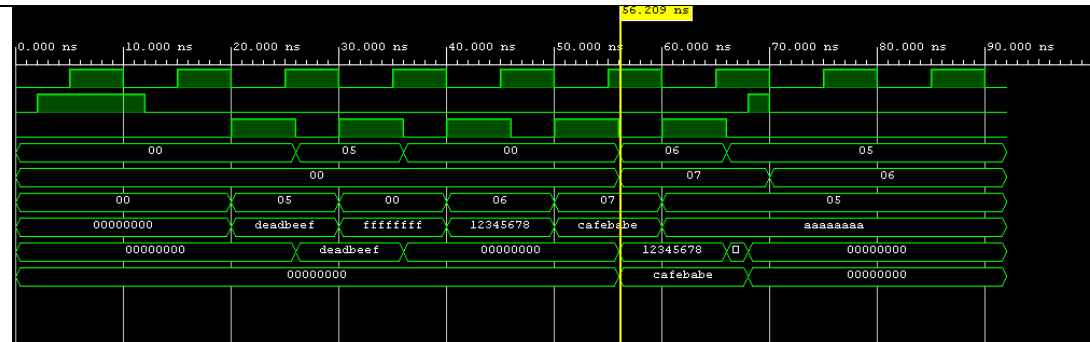
$finish;

end
```



Endmodule

Results:





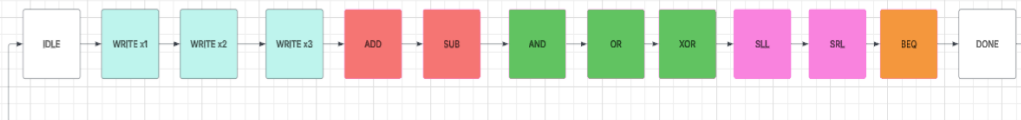
Task 2C

FSM testbench and FSM diagram



FSM Diagram:

TESTBENCH FSM DIAGRAM



RF ALU FSM tb.v

```
`timescale 1ns / 1ps
```

```
module RF_ALU_FSM_tb;
```

```
    reg clk;
```

```
    reg rst;
```

```
    reg WriteEnable;
```

```
    reg [4:0] rs1;
```

```
    reg [4:0] rs2;
```

```
    reg [4:0] rd;
```

```
    reg [31:0] WriteData;
```

```
    wire [31:0] ReadData1;
```

```
    wire [31:0] ReadData2;
```

```
    reg [2:0] ALUControl;
```

```
    wire [31:0] ALUResult;
```

```
    wire Zero;
```

```
    reg [3:0] state;
```

```
    reg [3:0] next_state;
```

```
    localparam IDLE = 4'h0;
```

```
    localparam WRITE_X1 = 4'h1;
```



```
localparam WRITE_X2 = 4'h2;
localparam WRITE_X3 = 4'h3;

localparam DO_ADD = 4'h4;
localparam DO_SUB = 4'h5;
localparam DO_AND = 4'h6;
localparam DO_OR = 4'h7;
localparam DO_XOR = 4'h8;
localparam DO_SLL = 4'h9;
localparam DO_SRL = 4'hA;
localparam DO_BEQ = 4'hB;

localparam DONE = 4'hC;

RegisterFile rf (
    .clk(clk),
    .rst(rst),
    .WriteEnable(WriteEnable),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd),
    .WriteData(WriteData),
    .ReadData1(ReadData1),
    .ReadData2(ReadData2)
);

ALU alu (

    .A(ReadData1),
    .B(ReadData2),
    .ALUControl(ALUControl),
    .Result(ALUResult),
    .Zero(Zero)
);

always #5 clk = ~clk;

// FSM state register
always @(posedge clk or posedge rst) begin

    if (rst)
```



```
state <= IDLE;

else
    state <= next_state;

end

// FSM control logic
always @(*) begin

    WriteEnable = 0;

    rs1 = 0;
    rs2 = 0;
    rd = 0;

    WriteData = 0;

    ALUControl = 3'b000;

    next_state = state;

    case (state)

        IDLE: begin
            next_state = WRITE_X1;
        end

        WRITE_X1: begin

            WriteEnable = 1;
            rd = 5'd1;

            WriteData = 32'h10101010;

            next_state = WRITE_X2;

        end

        WRITE_X2: begin
```



```
WriteEnable = 1;
rd = 5'd2;

WriteData = 32'h01010101;

next_state = WRITE_X3;

end
```

```
WRITE_X3: begin

    WriteEnable = 1;
    rd = 5'd3;

    WriteData = 32'h00000005;

    next_state = DO_ADD;

end
```

```
DO_ADD: begin

    rs1 = 5'd1;
    rs2 = 5'd2;

    ALUControl = 3'b000;

    WriteEnable = 1;
    rd = 5'd4;

    WriteData = ALUResult;

    next_state = DO_SUB;

end
```

```
DO_SUB: begin

    rs1 = 5'd1;
    rs2 = 5'd2;

    ALUControl = 3'b001;
```



```
WriteEnable = 1;
rd = 5'd5;

WriteData = ALUResult;

next_state = DO_AND;

end

DO_AND: begin

    rs1 = 5'd1;
    rs2 = 5'd2;

    ALUControl = 3'b010;

    WriteEnable = 1;
    rd = 5'd6;

    WriteData = ALUResult;

    next_state = DO_OR;

end

DO_OR: begin

    rs1 = 5'd1;
    rs2 = 5'd2;

    ALUControl = 3'b011;

    WriteEnable = 1;
    rd = 5'd7;

    WriteData = ALUResult;

    next_state = DO_XOR;

end
```



```
DO_XOR: begin
```

```
    rs1 = 5'd1;
```

```
    rs2 = 5'd2;
```

```
    ALUControl = 3'b100;
```

```
    WriteEnable = 1;
```

```
    rd = 5'd8;
```

```
    WriteData = ALUResult;
```

```
    next_state = DO_SLL;
```

```
end
```

```
DO_SLL: begin
```

```
    rs1 = 5'd1;
```

```
    rs2 = 5'd2;
```

```
    ALUControl = 3'b101;
```

```
    WriteEnable = 1;
```

```
    rd = 5'd9;
```

```
    WriteData = ALUResult;
```

```
    next_state = DO_SRL;
```

```
end
```

```
DO_SRL: begin
```

```
    rs1 = 5'd1;
```

```
    rs2 = 5'd2;
```

```
    ALUControl = 3'b110;
```

```
    WriteEnable = 1;
```

```
    rd = 5'd10;
```

```
    WriteData = ALUResult;
```



```
    next_state = DO_BEQ;

end

DO_BEQ: begin

    // Compare x1 with x1
    // Therefore Zero should become 1

    rs1 = 5'd1;
    rs2 = 5'd1;

    ALUControl = 3'b111;

    if (Zero) begin

        WriteEnable = 1;
        rd = 5'd11;

        WriteData = 32'h00000001;

    end

    next_state = DONE;

end

DONE: begin

    WriteEnable = 0;

    next_state = DONE;

end

default: begin

    next_state = IDLE;

end
```



```
endcase

end

initial begin

    clk = 0;
    rst = 0;

    #2;

    rst = 1;

    #10;

    rst = 0;

    wait(state == DONE);

    #2;

    if (rf.regs[1] == 32'h10101010)
        $display("PASS x1");

    if (rf.regs[2] == 32'h01010101)
        $display("PASS x2");

    if (rf.regs[3] == 32'h00000005)
        $display("PASS x3");

    if (rf.regs[4] == 32'h11111111)
        $display("PASS ADD");

    if (rf.regs[5] == 32'h0F0F0F0F)
        $display("PASS SUB");

    if (rf.regs[6] == 32'h00000000)
        $display("PASS AND");

    if (rf.regs[7] == 32'h11111111)
        $display("PASS OR");
```



```
if (rf.regs[8] == 32'h11111111)
    $display("PASS XOR");

if (rf.regs[9] == 32'h20202020)
    $display("PASS SLL");

if (rf.regs[10] == 32'h08080808)
    $display("PASS SRL");

if (rf.regs[11] == 32'h00000001)
    $display("PASS BEQ");

$display("-----");
$display("FSM + RF + ALU COMPLETE");
$display("-----");

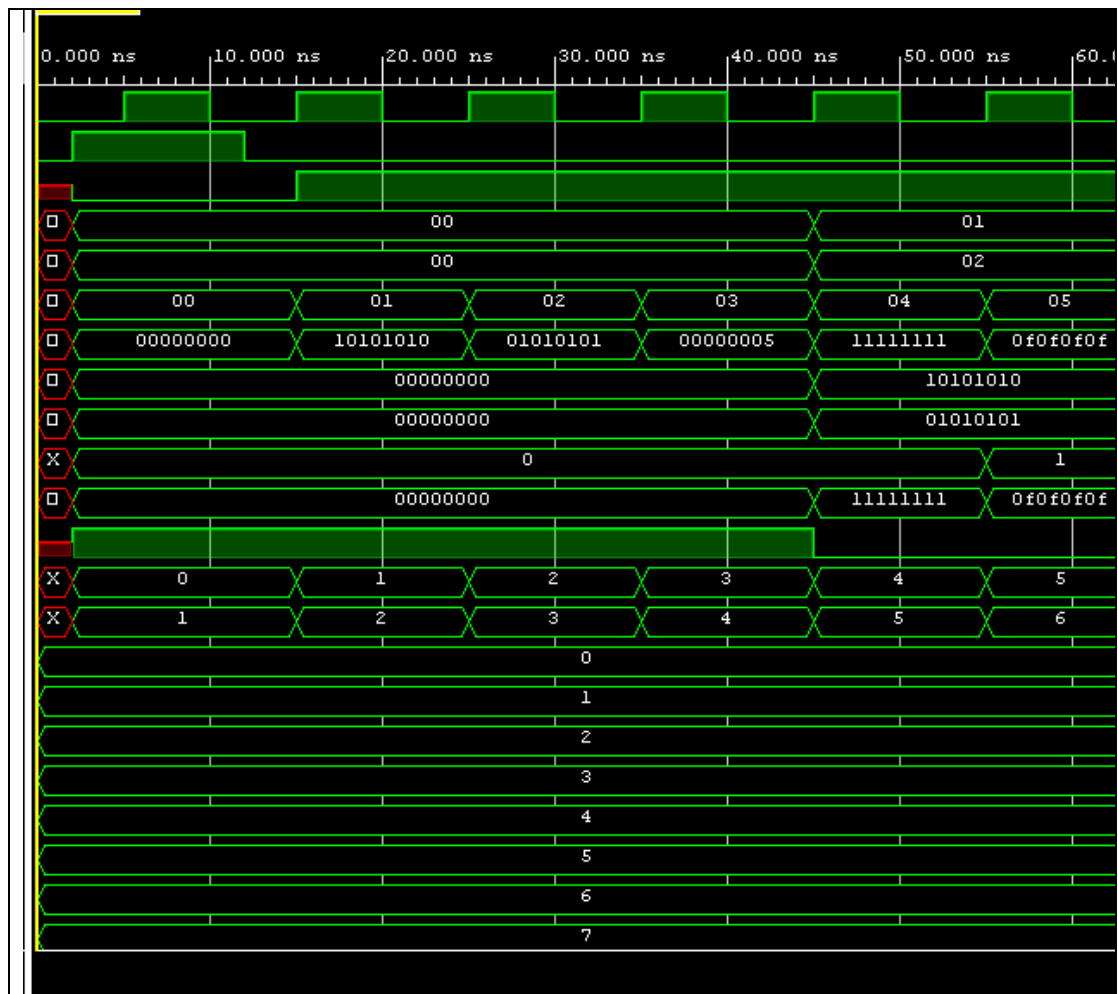
#20;

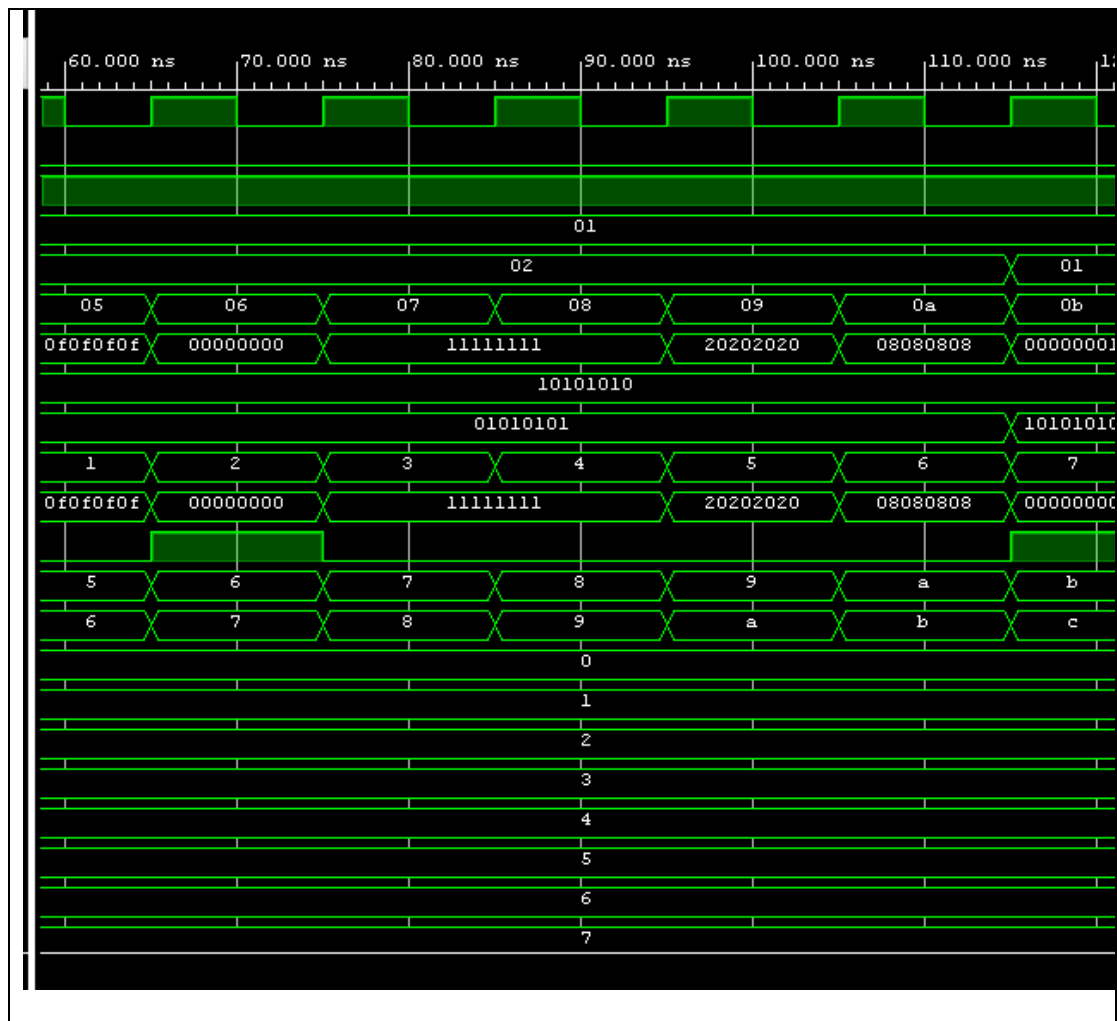
$finish;

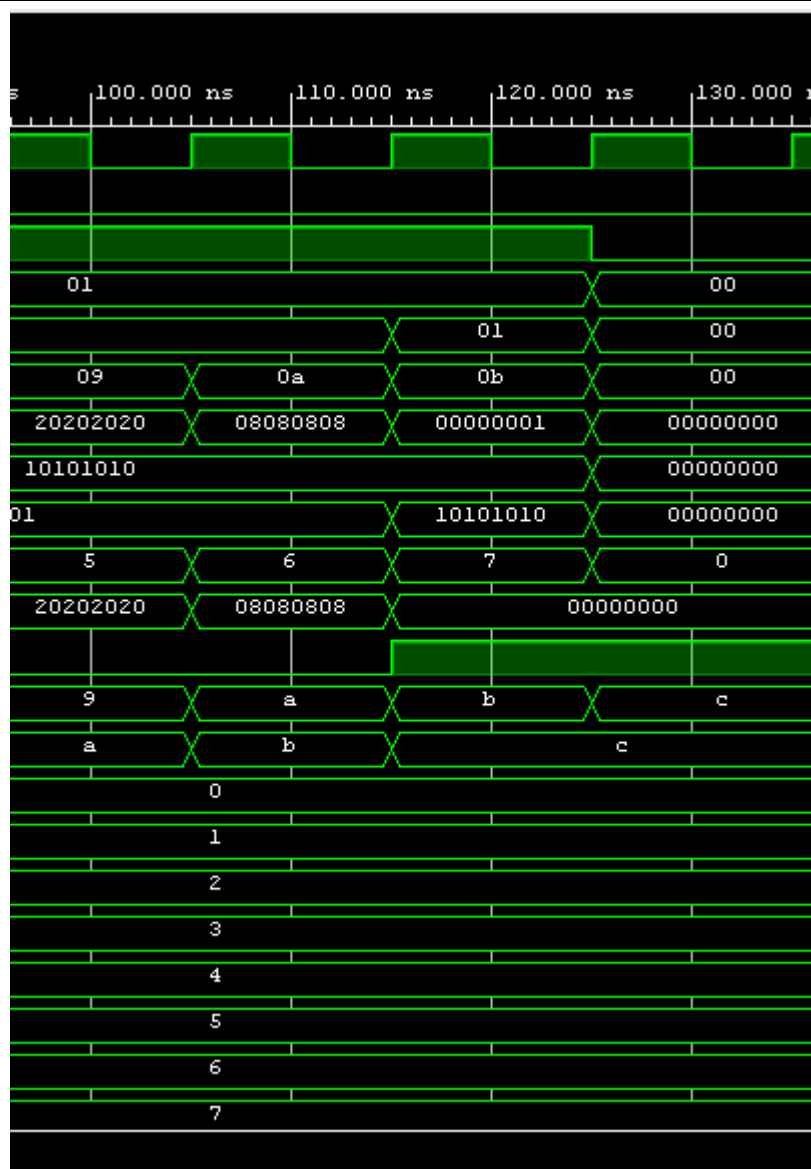
end

Endmodule
```

RESULTS:







As expected, results show that:

we wrote in x1 10101010, in x2 we wrote 01010101, in x3 we wrote 00000005. When we add x1 and x2, store the result in x4 we got 11111111. Subtraction of x2 from x1 showed us 0f0f0f0f in x5. AND operation of x1 and x2 got us 00000000. OR operation shows 11111111. XOR operation shows 11111111 as well.

SLL operation shows that we got 20202020 correctly, SRL shows we got 08080808. And last we compared x1 with x1 which is always true so BEQ condition becomes true and we see the result as 00000001



TASK 3:

TOP RF ALU design file:

```
module top_rf_alu (  
    input wire clk,  
    input wire btnC,  
    input wire btnU,  
    input wire [2:0] op_sel,  
    input wire write_enable_switch,  
    output wire [15:0] led,  
    output wire [6:0] seg,  
    output wire [3:0] an,  
    output wire dp  
);  
    wire step_pulse;  
    reg [3:0] state;  
    reg [3:0] next_state;  
    reg [4:0] rs1;  
    reg [4:0] rs2;  
    reg [4:0] rd;  
    reg [31:0] WriteData;  
    reg [2:0] ALUControl;  
    reg fsm_write_enable;  
    wire [31:0] ReadData1;  
    wire [31:0] ReadData2;  
    wire [31:0] ALUResult;  
    wire Zero;  
    wire actual_write_enable;  
    reg [7:0] display_data;  
  
    // =====  
    // STATES  
    // =====  
    localparam IDLE = 4'd0;  
    localparam WRITE_X1 = 4'd1;  
    localparam WRITE_X2 = 4'd2;  
    localparam WRITE_X3 = 4'd3;  
    localparam ADD_STATE = 4'd4;  
    localparam SUB_STATE = 4'd5;  
    localparam AND_STATE = 4'd6;  
    localparam OR_STATE = 4'd7;  
    localparam XOR_STATE = 4'd8;  
    localparam SLL_STATE = 4'd9;  
    localparam SRL_STATE = 4'd10;  
    localparam BEQ_STATE = 4'd11;
```



```
localparam DONE = 4'd12;
```

```
// =====  
// BUTTON  
// =====  
debounce_onepulse step_button (  
    .clk(clk),  
    .rst(btnC),  
    .button(btnU),  
    .pulse(step_pulse)  
);  
  
// =====  
// REGISTER FILE  
// =====  
assign actual_write_enable =  
    fsm_write_enable  
    & step_pulse  
    & write_enable_switch;  
  
RegisterFile rf (  
    .clk(clk),  
    .rst(btnC),  
    .WriteEnable(actual_write_enable),  
    .rs1(rs1),  
    .rs2(rs2),  
    .rd(rd),  
    .WriteData(WriteData),  
    .ReadData1(ReadData1),  
    .ReadData2(ReadData2)  
);  
  
// =====  
// ALU  
// =====  
ALU alu (  
    .A(ReadData1),  
    .B(ReadData2),  
    .ALUControl(ALUControl),  
    .Result(ALUResult),  
    .Zero(Zero)  
);  
  
// STATE REGISTER  
always @(posedge clk or posedge btnC) begin  
    if (btnC)  
        state <= IDLE;  
    else if (step_pulse)
```



```
state <= next_state;
end

// FSM
always @(*) begin
    rs1 = 5'd0;
    rs2 = 5'd0;
    rd = 5'd0;
    WriteData = 32'd0;
    ALUControl = 3'b000;
    fsm_write_enable = 1'b0;
    next_state = state;

    case (state)
    // IDLE
    IDLE: begin
        next_state = WRITE_X1;
    end

    // WRITE x1
    WRITE_X1: begin
        rd = 5'd1;
        WriteData = 32'h10101010;
        fsm_write_enable = 1'b1;
        next_state = WRITE_X2;
    end

    // WRITE x2
    WRITE_X2: begin
        rd = 5'd2;
        WriteData = 32'h01010101;
        fsm_write_enable = 1'b1;
        next_state = WRITE_X3;
    end

    // WRITE x3
    WRITE_X3: begin
        rd = 5'd3;
        WriteData = 32'h00000005;
        fsm_write_enable = 1'b1;
        next_state = ADD_STATE;
    end

    // ADD
    // x4 = x1 + x2
    ADD_STATE: begin
        rs1 = 5'd1;
        rs2 = 5'd2;
```



```
ALUControl = 3'b000;  
rd = 5'd4;  
WriteData = ALUResult;  
fsm_write_enable = 1'b1;  
next_state = SUB_STATE;  
end
```

```
// SUB  
SUB_STATE: begin  
rs1 = 5'd1;  
rs2 = 5'd2;  
ALUControl = 3'b001;  
rd = 5'd5;  
WriteData = ALUResult;  
fsm_write_enable = 1'b1;  
next_state = AND_STATE;  
end
```

```
// AND  
AND_STATE: begin  
rs1 = 5'd1;  
rs2 = 5'd2;  
ALUControl = 3'b010;  
rd = 5'd6;  
WriteData = ALUResult;  
fsm_write_enable = 1'b1;  
next_state = OR_STATE;  
end
```

```
// OR  
OR_STATE: begin  
rs1 = 5'd1;  
rs2 = 5'd2;  
ALUControl = 3'b011;  
rd = 5'd7;  
WriteData = ALUResult;  
fsm_write_enable = 1'b1;  
next_state = XOR_STATE;  
end
```

```
// XOR  
XOR_STATE: begin  
rs1 = 5'd1;  
rs2 = 5'd2;  
ALUControl = 3'b100;  
rd = 5'd8;  
WriteData = ALUResult;  
fsm_write_enable = 1'b1;
```



```
next_state = SLL_STATE;
end

// SLL
SLL_STATE: begin
  rs1 = 5'd1;
  rs2 = 5'd2;
  ALUControl = 3'b101;
  rd = 5'd9;
  WriteData = ALUResult;
  fsm_write_enable = 1'b1;
  next_state = SRL_STATE;
end

// SRL
SRL_STATE: begin
  rs1 = 5'd1;
  rs2 = 5'd2;
  ALUControl = 3'b110;
  rd = 5'd10;
  WriteData = ALUResult;
  fsm_write_enable = 1'b1;
  next_state = BEQ_STATE;
end

// BEQ
// x11 = 1 when x1 == x1
BEQ_STATE: begin
  rs1 = 5'd1;
  rs2 = 5'd1;
  ALUControl = 3'b111;
  rd = 5'd11;
  if (Zero)
    WriteData = 32'h00000001;
  else
    WriteData = 32'h00000000;
  fsm_write_enable = 1'b1;
  next_state = DONE;
end

// SW2:0 selects stored result.
// Press BTNU again -> loop to IDLE.
DONE: begin
  case (op_sel)
    3'b000: rs1 = 5'd4;
    3'b001: rs1 = 5'd5;
    3'b010: rs1 = 5'd6;
    3'b011: rs1 = 5'd7;
    3'b100: rs1 = 5'd8;
    3'b101: rs1 = 5'd9;
```



```
3'b110: rs1 = 5'd10;
3'b111: rs1 = 5'd11;
endcase
// LOOP BACK
next_state = IDLE;
end

default: begin
next_state = IDLE;
end
endcase
end

// LED DISPLAY DURING FSM
always @(*) begin
case (state)
ADD_STATE:
display_data = ALUResult[7:0];
SUB_STATE:
display_data = ALUResult[7:0];
AND_STATE:
display_data = ALUResult[7:0];
OR_STATE:
display_data = ALUResult[7:0];
XOR_STATE:
display_data = ALUResult[7:0];
SLL_STATE:
display_data = ALUResult[7:0];
SRL_STATE:
display_data = ALUResult[7:0];
BEQ_STATE:
display_data = Zero ? 8'h01 : 8'h00;
DONE:
display_data = ReadData1[7:0];
default:
display_data = ReadData1[7:0];
endcase
end

// LED OUTPUTS
assign led[7:0] = display_data;
assign led[11:8] = state;
assign led[12] = (state == DONE);
assign led[13] = Zero;
assign led[14] = actual_write_enable;
assign led[15] = write_enable_switch;
```



```
// 7 SEGMENT DISPLAY
SevenSegmentDisplay seven_seg (
.clk(clk),
.value(ReadData1[15:0]),
.seg(seg),
.an(an),
.dp(dp)
);
Endmodule
```

sevensegmentdecoder.v

```
module SevenSegmentDisplay (
input wire clk,
input wire [15:0] value,
output reg [6:0] seg,
output reg [3:0] an,
output wire dp
);
reg [19:0] refresh_counter = 20'd0;
wire [1:0] digit_select;
reg [3:0] hex_digit;

// =====
// REFRESH COUNTER
// =====
always @(posedge clk) begin
refresh_counter <= refresh_counter + 1'b1;
end

assign digit_select = refresh_counter[19:18];

// Decimal point OFF
assign dp = 1'b1;

// =====
// DIGIT SELECTION
//
// ANODES ARE ACTIVE LOW
// =====
always @(*) begin
case (digit_select)
2'b00: begin
// Right-most digit
an = 4'b1110;
hex_digit = value[3:0];
```



end

```
2'b01: begin
an = 4'b1101;
hex_digit = value[7:4];
end
```

```
2'b10: begin
an = 4'b1011;
hex_digit = value[11:8];
end
```

```
2'b11: begin
// Left-most digit
an = 4'b0111;
hex_digit = value[15:12];
end
```

```
default: begin
an = 4'b1111;
hex_digit = 4'h0;
end
endcase
end
```

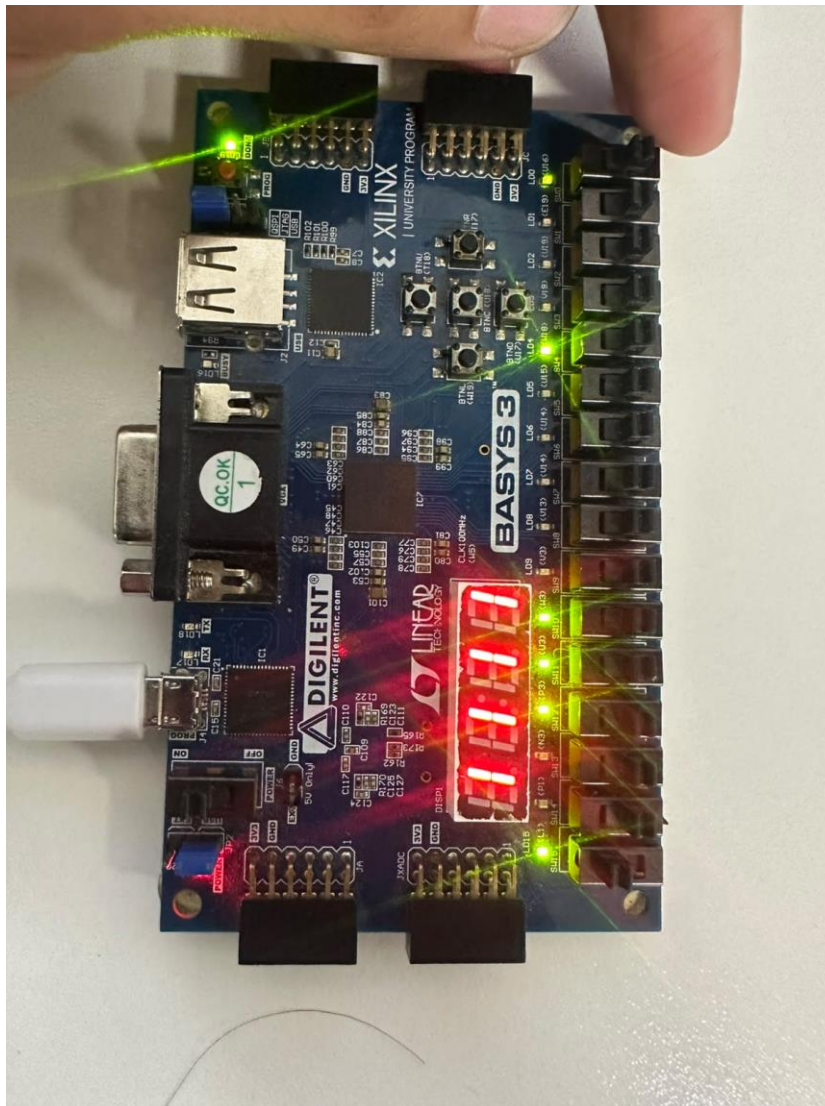
```
// =====
// HEX -> 7 SEGMENT DECODER
//
// seg[6:0] = G F E D C B A
//
// ACTIVE LOW
// 0 = segment ON
// 1 = segment OFF
// =====
```

```
always @(*) begin
case (hex_digit)
4'h0: seg = 7'b1000000;
4'h1: seg = 7'b1111001;
4'h2: seg = 7'b0100100;
4'h3: seg = 7'b0110000;
4'h4: seg = 7'b0011001;
4'h5: seg = 7'b0010010;
4'h6: seg = 7'b0000010;
4'h7: seg = 7'b1111000;
4'h8: seg = 7'b0000000;
4'h9: seg = 7'b0010000;
4'hA: seg = 7'b0001000;
4'hB: seg = 7'b0000011;
```

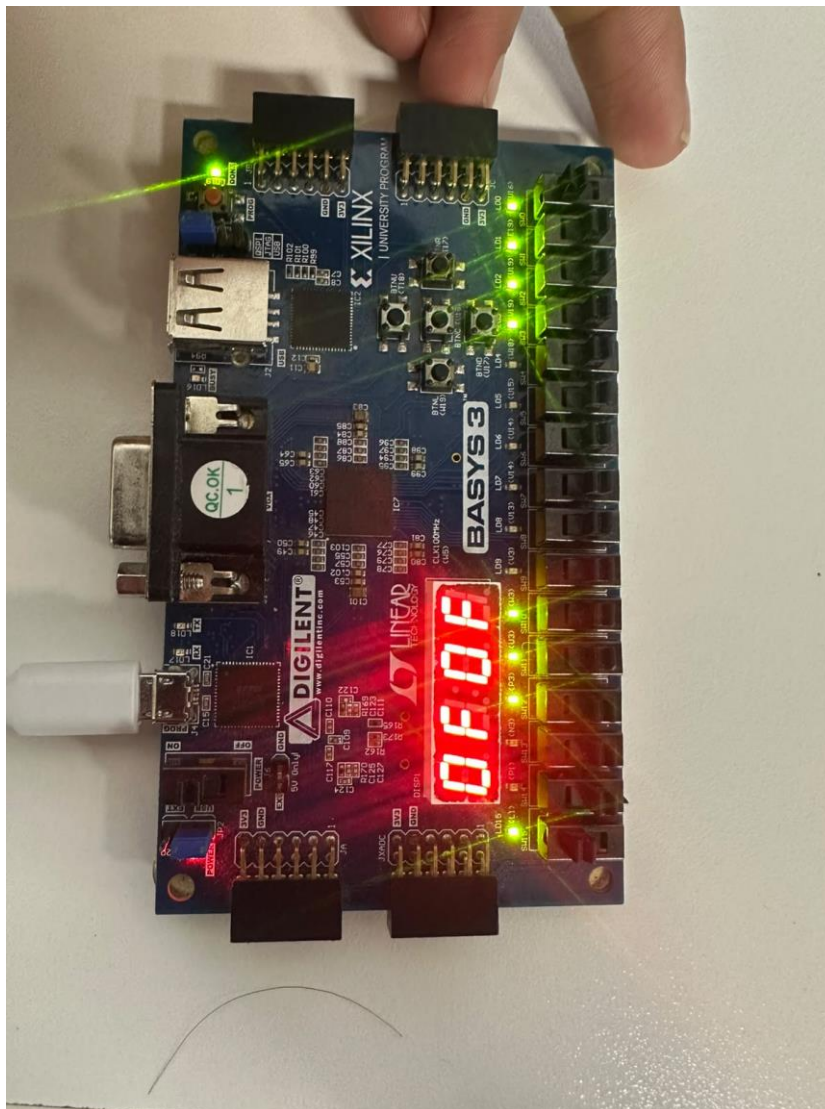


```
4'hC: seg = 7'b1000110;  
4'hD: seg = 7'b0100001;  
4'hE: seg = 7'b0000110;  
4'hF: seg = 7'b0001110;  
default:  
seg = 7'b1111111;  
endcase  
end  
endmodule  
RESULTS: A=1010 B=0101
```

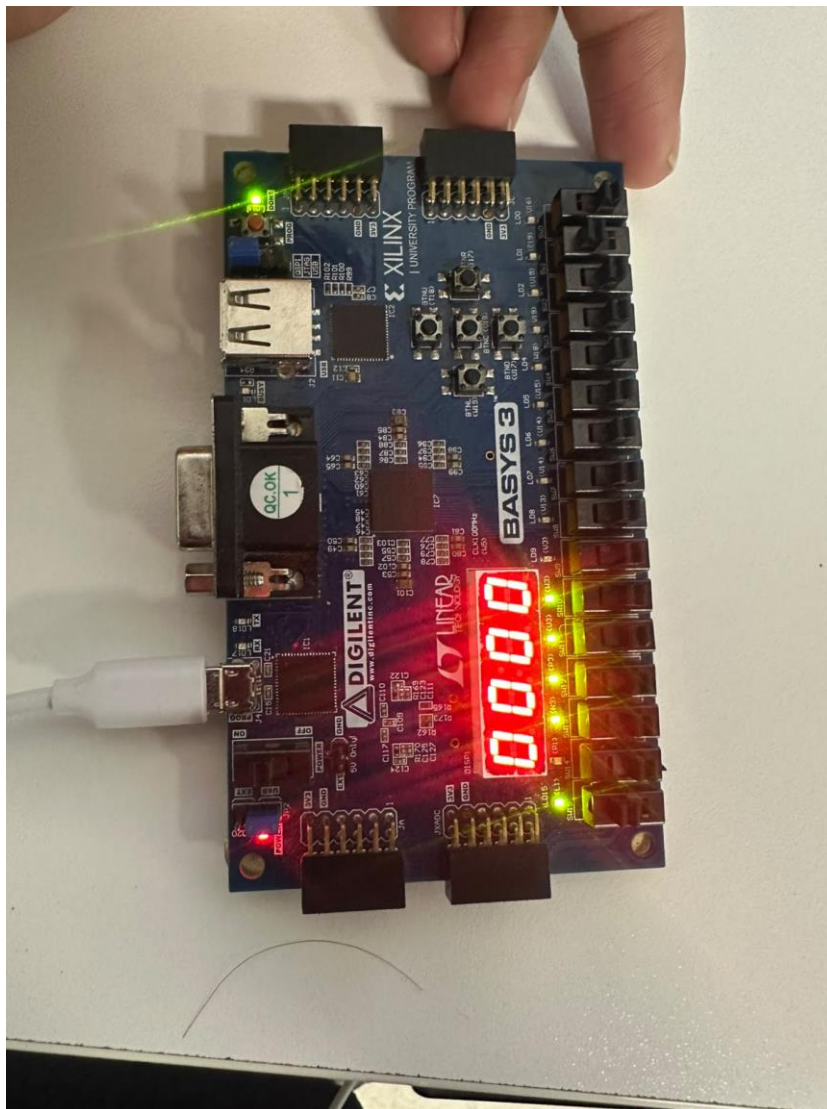
A ADD B



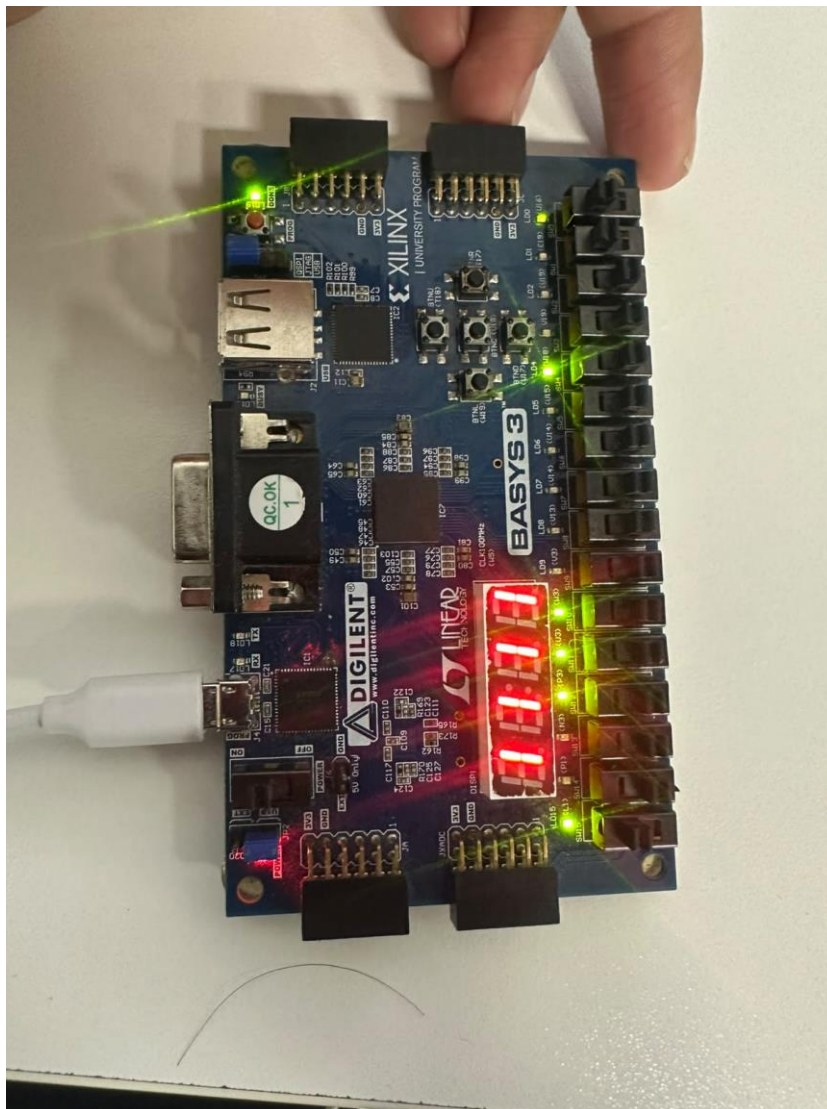
A SUB B



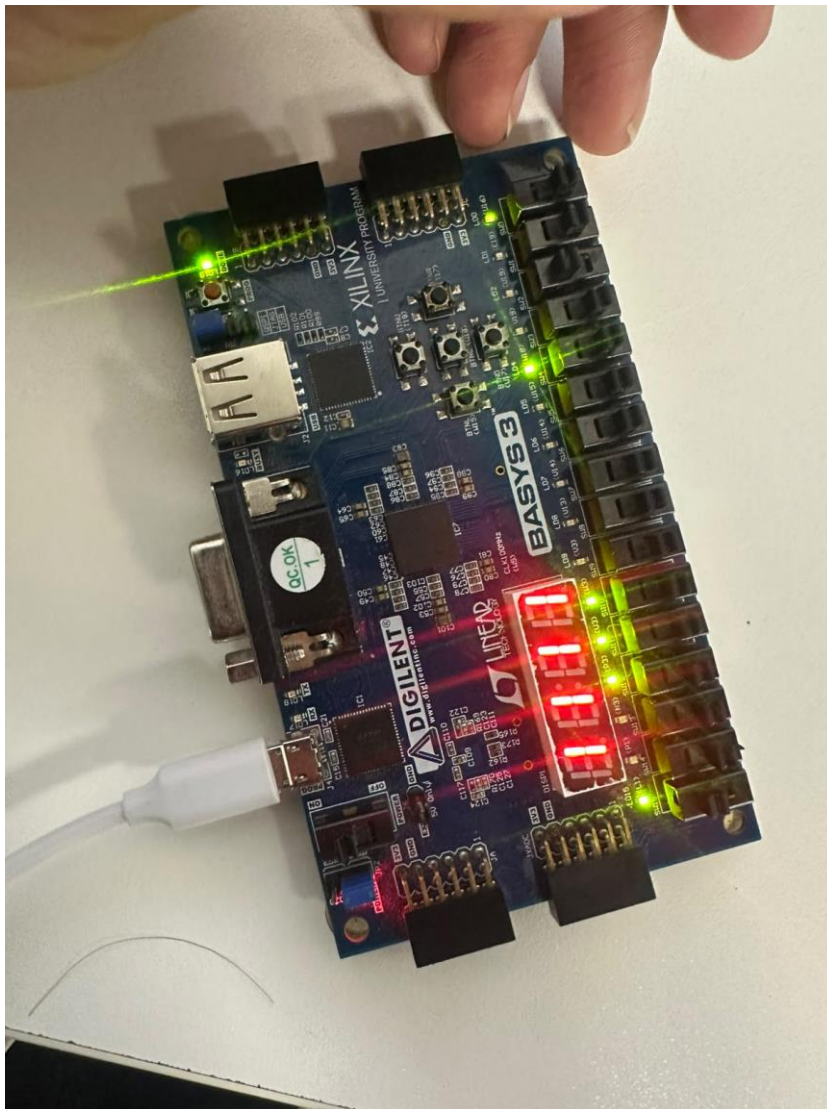
A AND B



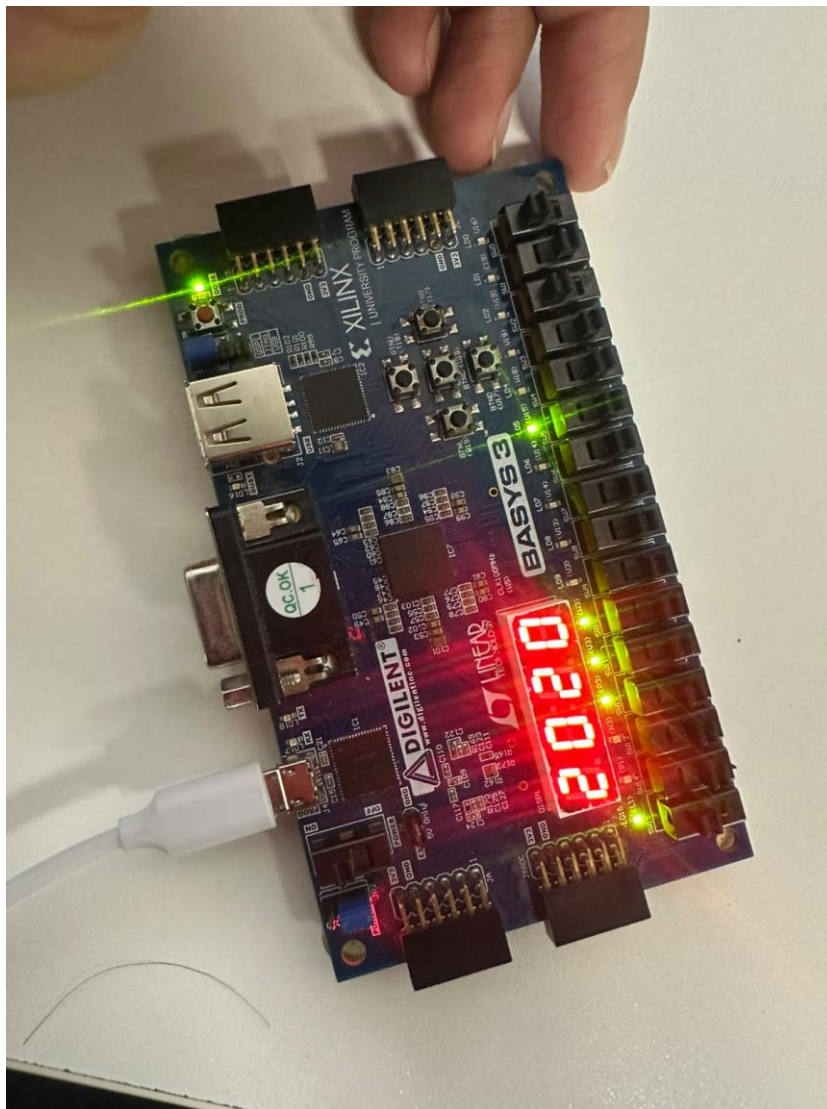
A OR B



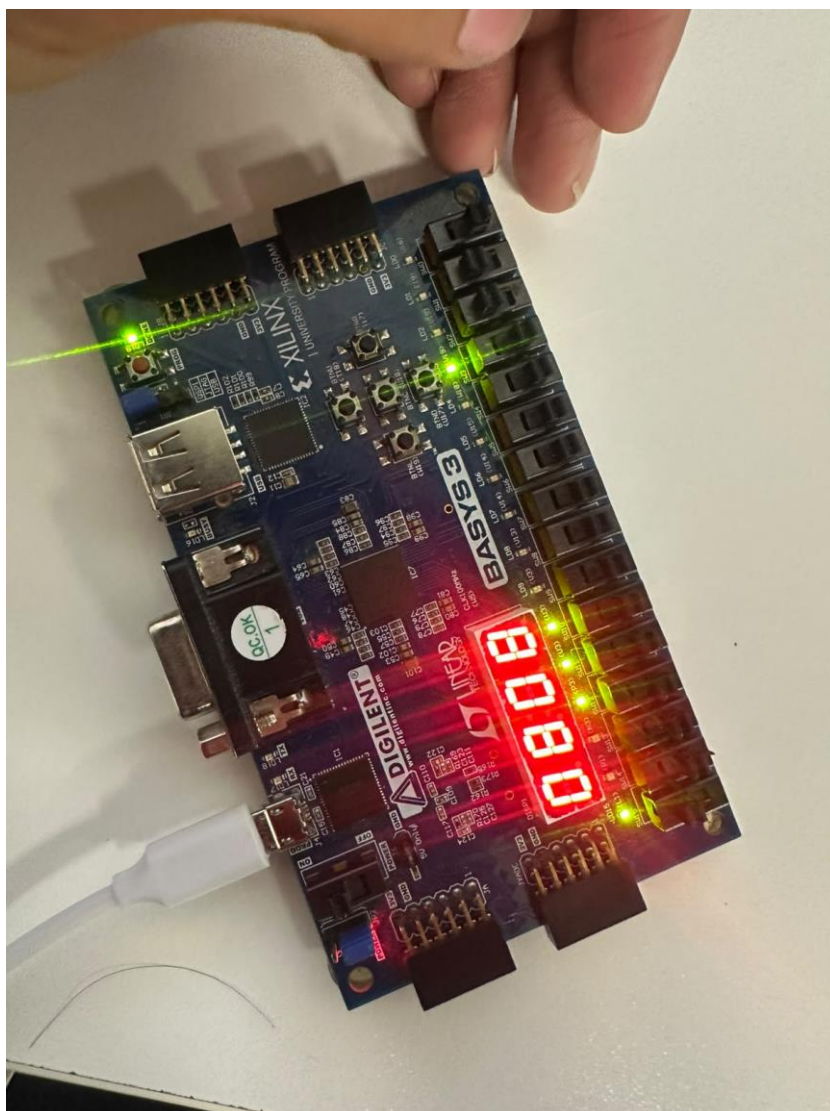
A XOR B



SLL ON A BY B



A SLR B



Lab 07

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Task 1	-	-	/20	/20
Task 2	/20	/15		
Task 3	/10	/15		
Total Points	/80			/20
CLO Mapped	CLO 1			



For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.